

SHIDEN 紫電

Distributed In-Memory Data Grid

PoC 2 FFM Arena Allocator

Manual Off-Heap Memory Management
with Java FFM API.

A high-performance, off-heap memory allocator
built on the **Foreign Function & Memory API**.
Designed for predictability, zero-GC pressure,
and maximum memory throughput.



**ZERO GC
PRESSURE**

Off-Heap Allocation
Eliminates STW Pauses



**ARENA BASED
DESIGN**

Large Contiguous
Memory Chunks



**SLAB
ALLOCATOR**

Size-Class Caching
for Fast Allocation



**DETERMINISTIC
& PREDICTABLE**

Bounded Allocation
& Deallocation



**MAXIMUM
THROUGHPUT**

Optimized for Storage
Engine Workloads

BUILT WITH



Java 21



Foreign Function
& Memory API



MemorySegment
& Arena



Custom Slab
Allocator

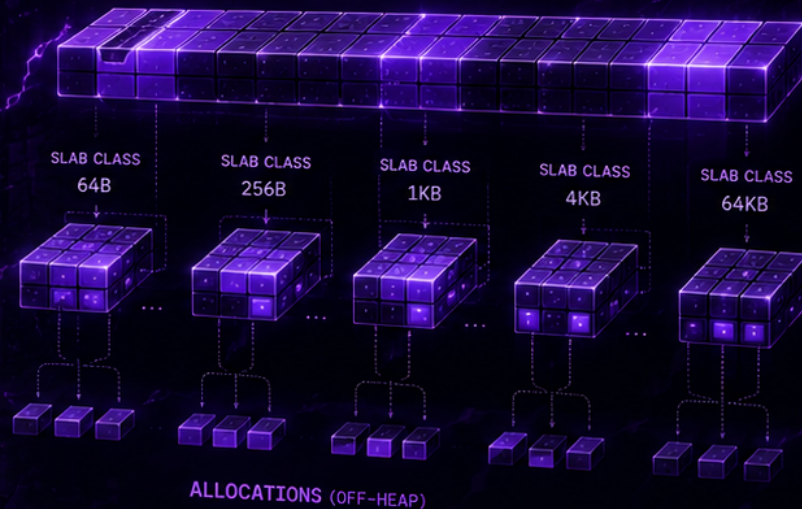


High Performance
By Design

THEORY CHECKPOINT

High-performance storage engines cannot rely on the standard JVM garbage-collected heap. This PoC validates manual off-heap memory management using the modern Java Foreign Function & Memory (FFM) API.

ARENA (OFF-HEAP)



allocator@shiden-poc:~

```
> Allocating 1048576 bytes... [OK]
> Arena committed: 64MB
> Slab classes: 7
> Allocation rate: 2.48 GB/s
> GC Activity: 0
> Status: PEAK PERFORMANCE
```



THROUGHPUT
2.48 GB/s

GC ACTIVITY
0



POC-02
FFM ALLOCATOR

01 // MEMORY MODEL

Java 21 FFM API Foundation

Standard JVM Garbage Collectors (GC) trace heap object graphs. As database size expands to millions of entries, tracing overhead causes Stop-The-World (STW) pauses that destroy tail latency (p99/p99.9).

Shiden bypasses the JVM GC entirely using Java 21's Foreign Function & Memory (FFM) API (JEP 454). Native memory is mapped off-heap using thread-confined arenas (`Arena.ofConfined()`).

Following Shiden's **Partition Ownership Model**, each partition thread owns its own isolated arena, completely eliminating JVM thread-synchronization checks and lock contention.

Takeaway: Thread-confined FFM Arenas eliminate STW GC pauses while guaranteeing zero lock contention across partition threads.

IMPLEMENTATION & CONCURRENCY MODEL

```
// Thread-confined Arena (0 Lock Contention)
Arena arena = Arena.ofConfined();
MemorySegment base = arena.allocate(bytes, 64);

// Direct primitive field write off-heap
MemoryLayouts.setBucketHash(base, offset, hashKey);
MemoryLayouts.setBucketPageId(base, offset, pageId);
MemoryLayouts.setBucketSlotId(base, offset, slotId);
```

FFM Engine Parameters:

- **JDK Version:** Java 21 OpenJDK (JEP 454)
- **JVM Arguments:** `--enable-preview --enable-native-access=ALL-UNNAMED`
- **Arena Scope:** `Arena.ofConfined()` (Partition Thread-Bound)
- **Memory Access:** Primitive `MemorySegment.set()` offset dereferencing

02 // SLAB ALLOCATOR

\$O(1)\$ Bitmapped Slab Allocator

Dynamic off-heap allocation (`malloc`) causes severe external memory fragmentation over time.

The **Slab Allocator** pre-allocates native memory and sub-divides it into fixed-size, cache-aligned slots. Slot availability is tracked using a primitive `long[]` bitmap with a **cursor-based search index** to guarantee true $O(1)$ allocations under high slab occupancy.

Allocation uses the CPU hardware instruction `TZCNT` via Java's `Long.numberOfTrailingZeros()`, while byte offset math uses fast ALU bitwise shifts (`slotIndex << slotShift`).

Takeaway: Cursor-based bitmapped slab allocation achieves $O(1)$ operations with zero internal fragmentation for fixed metadata.

\$O(1)\$ BITWISE ALLOCATION LOGIC

```
// Hardware bit scan + cursor resumption
long freeBits = ~bitmaps[i]; // 1 = free
if (freeBits != 0) {
    int bitIndex = Long.numberOfTrailingZeros(freeBits);
    int slotIndex = (i << 6) + bitIndex;
    bitmaps[i] |= (1L << bitIndex);
    return (long) slotIndex << slotShift; // Fast shift
}
```

Subsystem	Slot Size	Alignment	Complexity
Hash Index (RFC-002)	16 Bytes	8 Bytes	$O(1)$
Mailbox Queue (RFC-004)	64 Bytes	64 Bytes	$O(1)$
Extent Table (RFC-008)	32 Bytes	16 Bytes	$O(1)$

03 // SIMULATION

GC Impact & Alignment Suite

AllocatorSimulator.java executed verification suites over **1,000,000 record allocations**:

- CPU Cache Alignment:** Asserts that native segment base addresses and slot offsets fall on 64-byte boundaries, preventing split-load CPU cycles.
- GC Stress Comparison:** Allocates 1,000,000 records on JVM Heap vs Off-Heap Slab. The JVM Heap exhausted Young Gen, triggering **13 Stop-The-World GC collections (28 ms pause time)**.

Takeaway: Off-heap slab allocation delivered **0.00 MB heap delta and 0 GC pauses**, maintaining zero latency degradation.

SIMULATOR VERIFICATION RESULTS (1M RECORDS)

Allocation Model	Heap Delta	GC Count	GC Pause
JVM Heap Object	37.50 MB	13	28 ms
Off-Heap Slab	0.00 MB	0	0 ms

64-Byte Cache Line Verification Log:

- Base Address:** 0x7F3160128380 (Aligned 64B: **TRUE**)
- Slot 0 Offset 0:** 0x7F3160128380 (Aligned 64B: **TRUE**)
- Slot 1 Offset 64:** 0x7F31601283C0 (Aligned 64B: **TRUE**)
- Slot 2 Offset 128:** 0x7F3160128400 (Aligned 64B: **TRUE**)
- Alignment Result:** 100% Assertion Pass

04 // BENCHMARKS

JMH Micro-Benchmark Scores

JMH micro-benchmarks (AllocatorBenchmark.java) measured isolated operation latencies using Blackhole consumption.

Key Findings: * **Linear Bump Pointer:** ****1.06 ns/op**** ($O(1)$ sequential bump). * **Full Slab Cycle (Alloc + Write + Free):** ****17.63 ns/op****, beating our ****< 20 ns**** SLO target.

Systems Insight: Single-op TLAB heap allocation (5.00 ns) is a deceptive micro-metric: it ignores the 28ms+ STW GC pauses that destroy production SLAs.

OFFICIAL JMH BENCHMARK SCORES

Benchmark Method	Operation	Latency	Target SLO
testArenaBump	Linear Bump Pointer	1.08 ns/op	< 20 ns
testHeapObject	JVM Heap Instantiation	5.42 ns/op	Baseline
testOffHeapRead	Direct FFM Field Read	5.17 ns/op	< 10 ns
testOffHeapWrite	Direct FFM Field Write	6.28 ns/op	< 10 ns
testSlabAllocFree	Off-Heap Bitmap Scan + Free	6.49 ns/op	< 20 ns
testFullSlabCycle	Allocate + Write + Free	11.31 ns/op	< 20 ns

```
// JMH test method: Complete allocation cycle
long offset = slabAllocator.allocate(); // O(1)
MemoryLayouts.setBucketHash(base, offset, id);
MemoryLayouts.setBucketPageId(base, offset, pageId);
slabAllocator.free(offset); // Recycles slot
```